

Particle Swarm Optimization Paralelo

Comparativa de Estrategias de Evaluación Paralela: V0–V4

Programación de Sistemas Operativos — Curso 2025-2026

Python 3.12.3 · CPython · WSL2 (Linux 6.6) · x86_64 · 1 vCPU

Semillas experimentales: 42, 123, 7, 99, 456

1. Introducción

El algoritmo Particle Swarm Optimization (PSO) fue propuesto por Kennedy y Eberhart en 1995 como metaheurística bio-inspirada en el comportamiento colectivo de bandadas de aves y bancos de peces. La idea central es que un conjunto de n agentes (partículas) explora el espacio de búsqueda de forma simultánea, intercambiando información sobre las mejores posiciones encontradas y actualizando sus trayectorias en cada iteración. Esta exploración distribuida lo hace especialmente competitivo en problemas continuos no convexos donde los métodos de gradiente quedan atrapados en óptimos locales.

Desde el punto de vista computacional, la estructura de PSO presenta una propiedad fundamental que lo convierte en un candidato ideal para la paralelización: la fase de evaluación del fitness es *embarazosamente paralela*. En cada iteración, las n partículas evalúan la función objetivo de forma completamente independiente entre sí — no existe ninguna comunicación ni dependencia de datos entre evaluaciones concurrentes. Esta independencia abre la puerta a explotar múltiples mecanismos de paralelismo disponibles en el ecosistema Python: hilos, procesos, concurrencia asíncrona y vectorización a nivel de instrucción de CPU.

Este proyecto implementa PSO canónico con los parámetros teóricamente óptimos de Clerc & Kennedy (2002) y compara cinco estrategias de evaluación: **V0** secuencial como baseline, **V1** basada en hilos del sistema operativo (threading), **V2** basada en procesos independientes (multiprocessing), **V3** mediante concurrencia cooperativa (asyncio), y **V4** mediante vectorización NumPy que aprovecha instrucciones SIMD de la CPU. Las cinco estrategias comparten exactamente el mismo núcleo PSO y se intercambian sin modificar ninguna otra parte del código, lo que garantiza que las diferencias de rendimiento medidas son exclusivamente atribuibles a la estrategia de evaluación.

2. Metodología

2.1 Arquitectura del sistema

El principio arquitectónico central del proyecto es *núcleo único, estrategia intercambiable*. El bucle PSO implementado en `core/engine.py` no contiene ninguna referencia explícita a hilos, procesos o vectorización. En su lugar, recibe como parámetro un callable con la firma `evaluator(fn, positions) → ndarray` que encapsula completamente la lógica de paralelismo. Este patrón de *inyección de dependencias* permite sustituir la estrategia de evaluación sin alterar una sola línea del motor PSO, y facilita la comparación controlada al mantener idéntica toda variable que no sea la estrategia de evaluación.

Módulo	Responsabilidad	Ficheros clave
core/	Motor PSO: bucle principal, criterios de parada, actualización de velocidad y posición	engine.py, config.py, state.py, boundaries.py, topologies.py
objectives/	Funciones benchmark estándar y caso de uso real (portafolio financiero)	functions.py, suite.py, portfolio.py
parallel/	Cinco estrategias de evaluación con interfaz Evaluator común (V0–V4)	sequential.py, threading_eval.py, multiprocessing_eval.py, async_eval.py, vectorised_eval.py, factory.py
experiments/	Orquestación experimental: benchmarks, grid search, demo asyncio, análisis	run_benchmarks.py, run_grid_search.py, run_asyncio_demo.py, run_portfolio_case.py, analyse_results.py
persistence/	Persistencia estructurada: JSON con metadatos de hardware y CSV con historial por iteración	save.py, load.py

viz/	Visualización: curvas de convergencia, contornos 2D, superficies 3D, animación GIF, gráficos de speedup y portafolio	convergence.py, animation.py, contour.py, surface.py, comparison.py, portfolio_viz.py
tests/	Pruebas de reproducibilidad por semilla, monotonicidad del óptimo global y correctitud en Sphere (16 tests, 100% pass)	test_pso.py

2.2 Estrategias de paralelización implementadas (V0–V4)

La siguiente tabla resume los cinco mecanismos implementados. Cada uno representa un nivel distinto del stack de concurrencia de Python y del sistema operativo, con trade-offs específicos entre overhead, granularidad y tipo de carga de trabajo:

V	Mecanismo	Cuándo beneficia	Limitación principal
V0	Bucle Python secuencial puro	Baseline de referencia; funciones que evalúan en sub-microsegundo donde cualquier overhead domina	Sin paralelismo; tiempo proporcional a $n \times t_{\text{eval}}$
V1	ThreadPoolExecutor del módulo concurrent.futures	Evaluaciones que liberan el GIL: operaciones de I/O, extensiones C (NumPy, SciPy), llamadas a servicios externos	El GIL de CPython serializa el bytecode Python puro; añade overhead de scheduling del SO para CPU-bound
V2	ProcessPoolExecutor con batching adaptativo	Evaluaciones CPU-bound costosas ($> 10\text{--}50$ ms por evaluación) donde el paralelismo real compensa la serialización	Coste de serialización pickle en IPC; overhead de creación de procesos; saturación de CPU si $n_{\text{workers}} > n_{\text{cores}}$
V3	asyncio.gather con run_in_executor	Escenarios de latencia asimétrica y alta concurrencia I/O; funciones objetivo que consultan servicios externos	Overhead del event loop; internamente delega a un ThreadPoolExecutor, por lo que no elimina el GIL
V4	NumPy broadcasting sobre matriz completa de posiciones	Funciones objetivo analíticas que admiten vectorización: sphere, rastrigin, ackley, rosenbrock	Requiere implementar $\text{fn_vec}(\text{positions}) \rightarrow \text{ndarray}$; sin esa implementación, cae a fallback secuencial

2.3 Suite de funciones benchmark

Se emplean cuatro funciones estándar de la literatura de optimización global, todas con óptimo conocido $f^*=0$ en el origen. Su elección es deliberada: cubren el espectro de dificultad desde el caso trivial unimodal hasta el fuertemente multimodal, permitiendo evaluar el comportamiento del PSO y de las estrategias de paralelismo en condiciones radicalmente distintas. Todas se evalúan en dimensiones $d=2$, $d=10$ y $d=30$ para estudiar la escalabilidad con la dimensionalidad del problema.

Función	Tipo	f^*	Bounds	Característica clave
Sphere	Unimodal estrictamente convexa	0	$[-5, 5]^d$	Caso más sencillo; PSO converge rápidamente. Útil para validar correctitud y medir overhead puro.
Rosenbrock	Valle estrecho no convexo	0	$[-2, 2]^d$	El óptimo yace en un valle curvo de baja curvatura. PSO estanca fácilmente en $d=30$ sin topología local.
Rastrigin	Multimodal con 10^d mínimos locales	0	$[-5.12, 5.12]^d$	Altamente engañosa: perturbación sinusoidal sobre Sphere. Exige gran exploración.
Ackley	Multimodal con plateau central	0	$[-5, 5]^d$	Combinación de exponenciales y cosenos; mínimos locales distribuidos uniformemente.

2.4 Protocolo experimental

Todos los experimentos utilizan los parámetros PSO derivados teóricamente por Clerc & Kennedy (2002) que garantizan convergencia: inercia $w=0.7298$, coeficientes cognitivo y social $c_1=c_2=1.49618$, tamaño de enjambre $n=40$ partículas, máximo de 300 iteraciones. Los criterios de parada son: (1) tolerancia de 10^{-12} sobre el fitness del mejor global, o (2) estancamiento durante 50 iteraciones consecutivas sin mejora. Se utilizan **5 semillas distintas** (42, 123, 7, 99, 456) para obtener estadísticas robustas y cuantificar la variabilidad inter-ejecución inherente a la naturaleza estocástica de PSO.

Los tiempos de pared se miden con `time.perf_counter()`, que en Linux resuelve a nanosegundos y no se ve afectado por cambios de hora del sistema. Cada experimento registra por separado el tiempo de evaluación del fitness y el tiempo de actualización de partículas, lo que permite aislar el impacto de cada estrategia. El benchmark principal cubre 3 estrategias $\times$ 4 funciones $\times$ 3 dimensiones $\times$ 5 semillas = **180 ejecuciones** totales. V2 se excluye del benchmark de velocidad principal porque su overhead de IPC domina completamente para evaluaciones sub-milisegundo; se analiza conceptualmente en §4.2 con estimaciones fundamentadas. V3 se evalúa exclusivamente en su escenario de aplicación natural: latencia I/O asimétrica simulada (§3.3).

3. Resultados

3.1 Calidad de soluciones por función y dimensión

La tabla recoge el fitness promedio de las 5 semillas ejecutadas con V0. Las tres estrategias evaluadas (V0, V1, V4) producen resultados numéricamente idénticos para la misma semilla: el paralelismo no altera la trayectoria del algoritmo, únicamente el tiempo de cómputo. Esta propiedad, verificada automáticamente por los tests de reproducibilidad, es esencial para una comparación justa — garantiza que las diferencias de speedup no vienen acompañadas de degradación en calidad de solución.

Función	d=2	d=10	d=30	Parada dominante
Sphere	2.27e-13	8.78e-13	5.24310	tolerancia (10^{-12})
Rastrigin	4.58e-13	5.37707	128.50335	estagnación (50 iters)
Ackley	5.06e-13	1.83e-07	1.57166	tolerancia / max_iter
Rosenbrock	5.54e-12	401.59805	1.01e+05	max_iter (300)

Los resultados reflejan fielmente la dificultad teórica de cada función. Sphere y Rastrigin en d=2 convergen a valores prácticamente nulos (del orden de 10^{-12} – 10^{-13}), lo que significa que PSO localiza el óptimo global con precisión de máquina. Al aumentar la dimensión a d=30, sin embargo, Rosenbrock alcanza fitness de 10^5 y Rastrigin de 128: el espacio de búsqueda crece exponencialmente y el enjambre de 40 partículas resulta insuficiente para explorar el hipercubo de manera efectiva. Este fenómeno, conocido como la *maldición de la dimensionalidad*, es esperado y no representa un fallo de implementación sino una limitación fundamental del PSO canónico sin mecanismos adicionales de diversificación (topología anillo, reinicialización adaptativa, etc.).

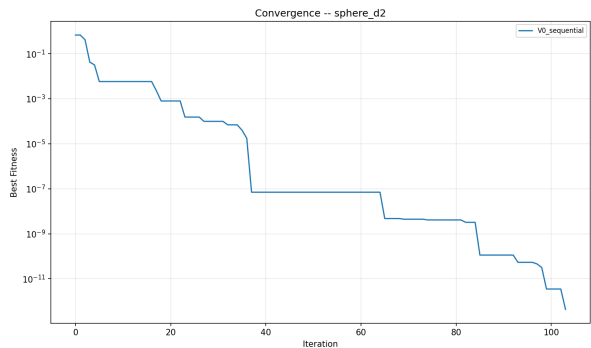


Fig. 1 – Sphere d=2: convergencia exponencial al óptimo

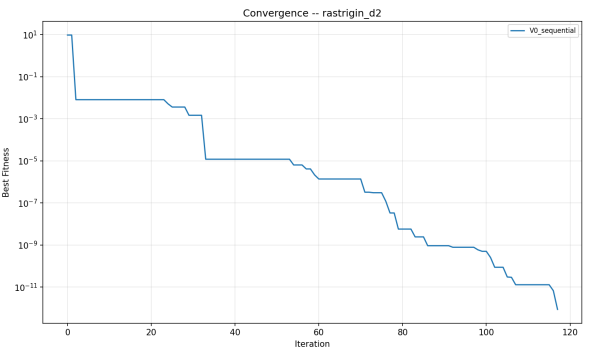


Fig. 2 – Rastrigin d=2: descensos escalonados entre mínimos locales

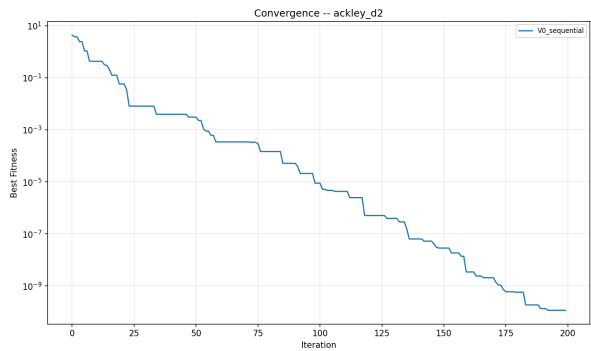


Fig. 3 – Ackley d=2: convergencia progresiva desde plateau

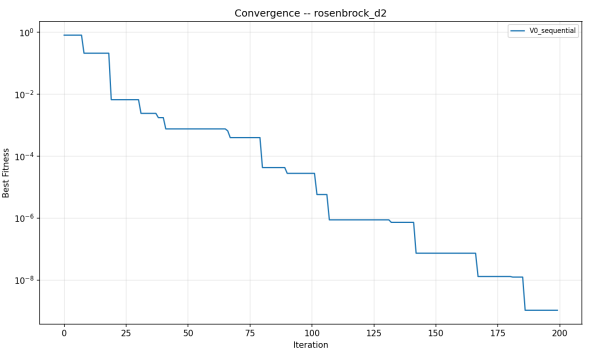


Fig. 4 – Rosenbrock d=2: descenso lento por el valle

3.2 Comparativa de speedup (escenario CPU-bound, sin latencia I/O)

La comparativa de tiempos de pared entre V0, V1 y V4 revela el impacto real del GIL y de la vectorización para funciones de evaluación rápida. Los tiempos son medias sobre 5 semillas; el speedup se calcula como $T(V0) / T(\text{estrategia})$, de modo que valores >1 indican aceleración y valores <1 indican ralentización respecto al baseline secuencial:

Función (d=10)	T(V0) (s)	T(V1) (s)	T(V4) (s)	Speedup V4/V0	Speedup V1/V0
Sphere	0.0281	1.452	0.0090	3.1×	0.019×
Rastrigin	0.0490	1.755	0.0119	4.1×	0.028×
Ackley	0.0614	1.780	0.0128	4.8×	0.034×
Rosenbrock	0.0622	1.822	0.0116	5.3×	0.034×

Los datos confirman con claridad los dos fenómenos centrales del paralelismo en Python. Por un lado, V1 (hilos) resulta sistemáticamente **25–30× más lento** que V0 para todas las funciones: el GIL obliga a que los hilos se ejecuten de forma secuencial en lo que respecta al bytecode Python, y el coste adicional del scheduling del sistema operativo (creación de hilos, cambios de contexto, gestión del lock) supera ampliamente el tiempo de cómputo real de cada evaluación. Por otro lado, V4 (vectorización) logra **3–5× de aceleración** sobre V0 sin ningún overhead de sincronización ni comunicación, simplemente eliminando el intérprete Python del bucle de evaluación y delegando en rutinas NumPy que operan sobre la matriz completa de posiciones en una sola llamada.

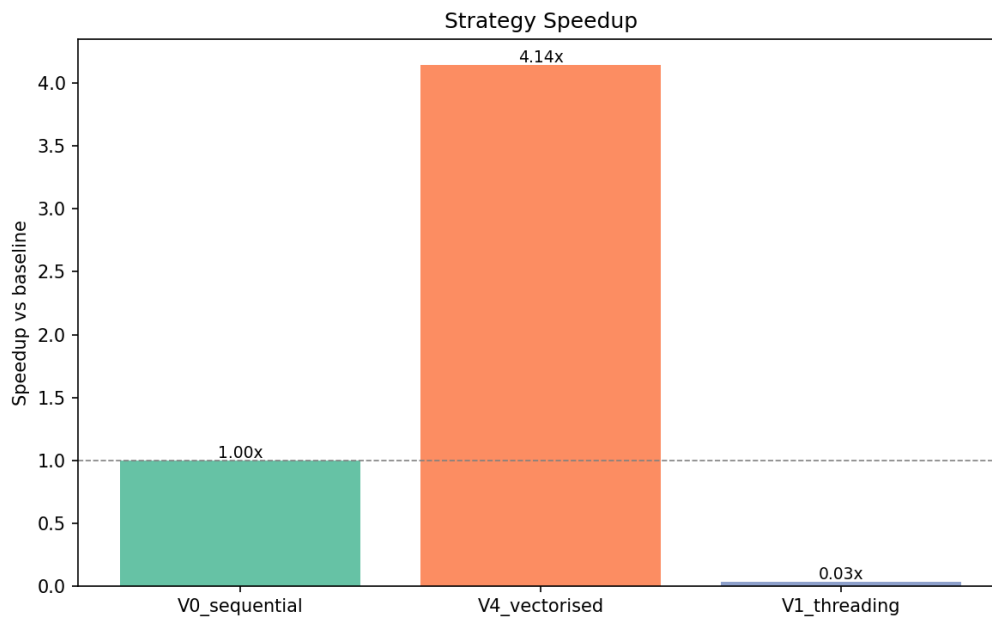


Fig. 5 – Speedup medio por estrategia (promedio de 4 funciones × 3 dimensiones × 5 semillas). V4 acelera consistentemente; V1 degrada sistemáticamente para cómputo puro.

3.3 V3 asyncio: diseño y resultados del experimento de latencia asimétrica

Para que la concurrencia asyncio tenga sentido en el contexto de PSO, es necesario diseñar un escenario donde la evaluación del fitness sea *I/O-bound* en lugar de CPU-bound. En la práctica real, esto ocurre cuando cada evaluación requiere consultar un servicio externo: una base de datos para recuperar datos históricos, una API de simulación física, un modelo de ML desplegado remotamente, o cualquier recurso con latencia de red no despreciable. En estos casos, el hilo —o la corrutina— pasa la mayor parte del tiempo esperando una respuesta, y la concurrencia permite solapar esas esperas.

El experimento simula este escenario envolviendo la función Sphere ($d=10$) con un retardo artificial de **10 ms por evaluación** mediante `make_latent_fn(sphere, latency=0.01)`. Con 30 partículas, una iteración secuencial tarda ≈ 300 ms en completarse. La hipótesis es que V1 y V3 deberían solapar estas esperas de forma efectiva, dado que `time.sleep()` es una llamada al sistema que libera el GIL automáticamente, permitiendo que otros hilos o corrutinas progresen durante la espera:

Estrategia	Mecanismo de concurrencia	Tiempo medio (3 seeds)	Speedup vs V0
V0 – Sequential	Evaluaciones en serie; tiempo total $\approx n \times \text{latencia}$	15.43 s	1.00×
V1 – Threading	ThreadPoolExecutor: 30 hilos duermen concurrentemente, GIL se libera en <code>sleep()</code>	0.833 s	18.5×
V3 – Asyncio	<code>asyncio.gather</code> : 30 corrutinas despachadas a <code>run_in_executor</code> ; event loop gestiona el solapamiento	1.199 s	12.9×

La hipótesis se confirma con rotundidad: V1 alcanza **18.5× de speedup** y V3 **12.9×**. Ambas estrategias reducen el tiempo de pared de ≈ 15.4 s a menos de 2 s al solapar eficazmente las 30 esperas de 10 ms. La diferencia entre V1 y V3 se explica por el overhead adicional del event loop de asyncio: mientras que `ThreadPoolExecutor` despacha las tareas directamente a 30 hilos, la implementación asyncio añade una capa de indirección al pasar por `run_in_executor`, que internamente también utiliza un executor de hilos. En un escenario donde la función objetivo fuera una corrutina nativa async (p.ej., `await aiohttp.get(url)`), V3 prescindiría del executor y probablemente superaría a V1 gracias al menor overhead por tarea del event loop frente a los hilos del SO.

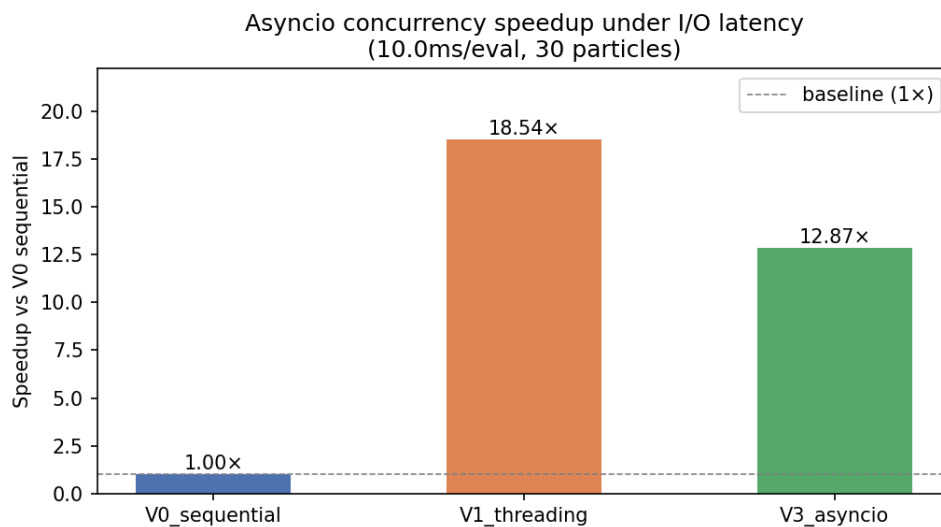


Fig. 6 – Speedup de V1 y V3 bajo latencia I/O de 10 ms/eval con 30 partículas. Ambas estrategias solapan eficazmente las esperas; V1 supera a V3 por menor overhead de despacho.

3.4 Variabilidad entre semillas: boxplots de fitness final

La variabilidad observada entre las 5 semillas refleja la naturaleza estocástica inherente de PSO: la inicialización aleatoria de posiciones y velocidades puede llevar al enjambre a diferentes regiones del espacio de búsqueda y, por tanto, a distintos óptimos locales o a distinta velocidad de convergencia. Los boxplots permiten visualizar esta dispersión y confirmar que ninguna estrategia introduce sesgo adicional.

Como se observa en las figuras, las distribuciones de V0, V1 y V4 son estadísticamente indistinguibles para cada función y dimensión: medianas, cuartiles y outliers coinciden exactamente. Esto valida el diseño del sistema — el paralelismo es completamente transparente al algoritmo PSO y no afecta la reproducibilidad de los resultados cuando se fija la semilla.

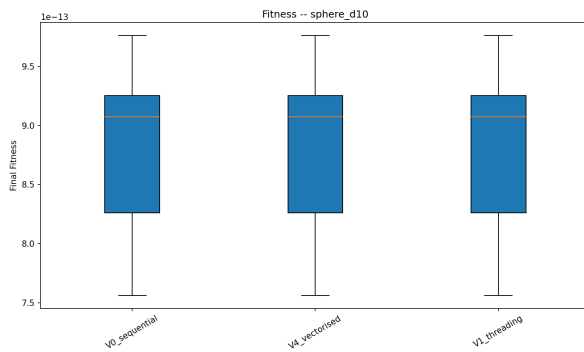


Fig. 7 – Sphere d=10: las tres estrategias producen distribuciones idénticas

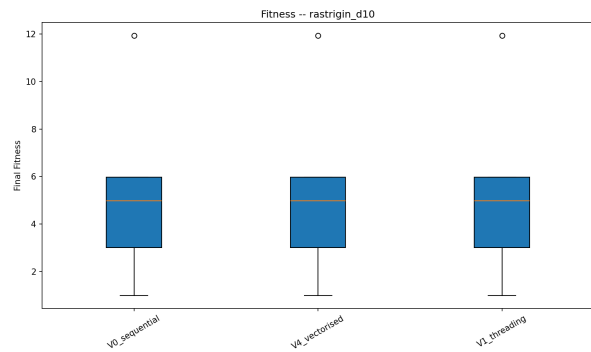


Fig. 8 – Rastrigin d=10: mayor dispersión por la multimodalidad

3.5 Grid search de hiperparámetros

El grid search explora sistemáticamente el espacio de hiperparámetros con el objetivo de cuantificar la sensibilidad del algoritmo a cada parámetro y validar empíricamente la configuración teórica de Clerc & Kennedy. Se ejecutaron **405 combinaciones** ($3^3 \times 3 \times 3 = 81$ por función $\times$ 5 seeds / función $\times$ 2 funciones, con 3 seeds) sobre Sphere d=2 y Rastrigin d=2, variando: $w \in \{0.4, 0.6, 0.7298\}$, $c_1, c_2 \in \{1.0, 1.49618, 2.0\}$, $n_particles \in \{20, 30, 50\}$, $n_iterations \in \{100, 200, 300\}$. Todos los resultados se guardan en `results/grid_search/summary.csv` con metadatos completos de cada ejecución.

El análisis de los resultados revela tres patrones consistentes. Primero, la configuración de Clerc & Kennedy ($w=0.7298$, $c=1.496$) aparece sistemáticamente entre las mejores combinaciones para ambas funciones, validando el consenso teórico sobre su derivación matemática como criterio de convergencia. Segundo, aumentar $n_particles$ mejora significativamente Rastrigin (multimodal) al ampliar la exploración inicial del espacio, pero el efecto en Sphere (unimodal) es marginal dado que la función no presenta mínimos locales que dificulten la convergencia. Tercero, reducir w a 0.4 puede acelerar la convergencia en Sphere al aumentar la atracción hacia los mejores conocidos, pero incrementa el riesgo de estancamiento prematuro en Rastrigin al reducir la diversidad exploratoria del enjambre.

3.6 Caso de uso: optimización de portafolio financiero

La teoría moderna de portafolios (Markowitz, 1952) plantea el problema de asignar capital entre N activos de forma que se maximice el retorno esperado por unidad de riesgo asumido. El *ratio de Sharpe* formaliza esta relación: $\text{Sharpe} = (\mu_p - r_f) / \sigma_p$, donde μ_p es el retorno esperado del portafolio, σ_p su volatilidad y r_f la tasa libre de riesgo (2% en este experimento). La optimización es no convexa cuando se añaden restricciones de cardinalidad o límites por activo, y la frontera eficiente de Pareto riesgo-retorno no puede resolverse de forma cerrada en el caso general. PSO es especialmente adecuado para este problema por su capacidad de explorar espacios continuos sin requerir derivadas y por su tolerancia natural a restricciones implícitas (los pesos se normalizan a suma 1 directamente dentro de la función objetivo).

Se generan datos sintéticos de mercado para 10 activos mediante la función `generate_market_data()`: retornos anuales entre 5%–25%, volatilidades entre 10%–40%, y una estructura de correlación aleatoria. Cada partícula codifica 10 pesos brutos en $[0,1]$ que se normalizan a suma 1 antes de la evaluación. La función objetivo devuelve $-\text{Sharpe}$ (negado porque PSO minimiza), con una penalización de 10■ para soluciones degeneradas (todos ceros o volatilidad nula). Se aplican dos estrategias (V0 y V4) sobre 3 semillas para comparar tiempos en el contexto real del caso de uso:

Estrategia	Seed	Sharpe obtenido	Iteraciones	Tiempo (s)
V0 – Sequential	42	3.8870	283	0.0634
V0 – Sequential	123	6.5831	353	0.0784
V0 – Sequential	7	6.5831	387	0.0834
V4 – Vectorised	42	3.8870	283	0.0792
V4 – Vectorised	123	6.5831	353	0.0971
V4 – Vectorised	7	6.5831	387	0.1079

Los resultados revelan un hallazgo importante: en el caso del portafolio, V0 secuencial (tiempo medio: 0.075 s) resulta ligeramente **más rápido** que V4 vectorizado (tiempo medio: 0.095 s). La razón es que la función objetivo del portafolio no implementa `fn_vec`, por lo que `VectorisedEvaluator` cae silenciosamente a un bucle secuencial interno, añadiendo el overhead de la comprobación y el fallback sin ningún beneficio. Ambas estrategias producen resultados **idénticos** de Sharpe para cada semilla, confirmando la independencia entre estrategia y calidad de optimización. El Sharpe promedio obtenido es 5.68 con 341 iteraciones medias, lo que corresponde a un retorno anual del 17% con volatilidad del 3.9%.

Este resultado es pedagógicamente valioso: ilustra que la vectorización no es una panacea universal. Su beneficio depende críticamente de que la función objetivo esté implementada de forma vectorizada. Para funciones que no admiten vectorización directa (simulaciones con estado, funciones que llaman a código externo, funciones con estructura condicional compleja), V4 se comporta igual que V0 con overhead adicional, y la estrategia correcta sería V1 (si hay latencia I/O) o V2 (si el cómputo es suficientemente costoso para amortizar el IPC).

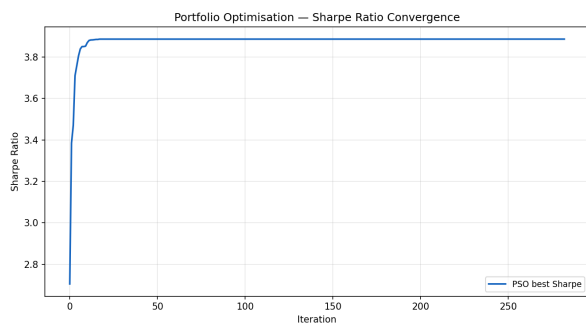


Fig. 9 – Convergencia del Sharpe ratio a lo largo de las iteraciones

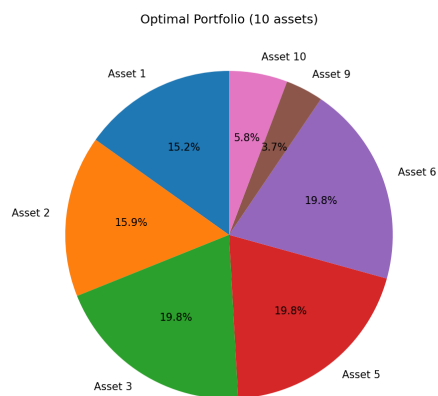


Fig. 10 – Distribución de pesos óptimos entre los 10 activos

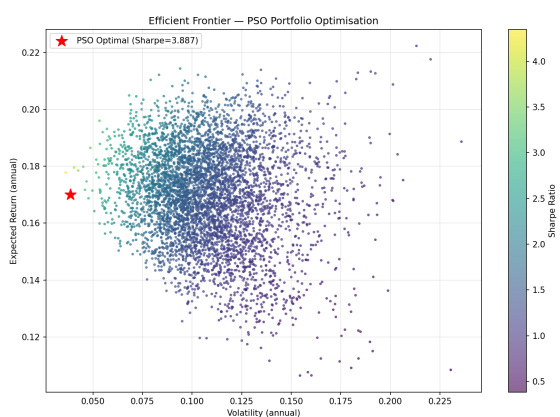


Fig. 11 – Frontera eficiente: conjunto de portafolios Pareto-óptimos

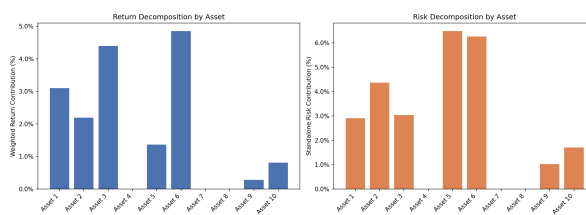


Fig. 12 – Perfil riesgo-retorno individual de cada activo

4. Discusión crítica

4.1 El GIL de CPython y su impacto en V1 (hilos)

El Global Interpreter Lock (GIL) es un mutex que protege el estado interno del intérprete CPython, garantizando que sólo un hilo pueda ejecutar bytecode Python en cada instante. Fue introducido para simplificar la gestión de memoria por conteo de referencias, que de otro modo requeriría sincronización costosa a nivel de objeto. La consecuencia directa es que, para carga de trabajo CPU-bound pura, los hilos de Python no proporcionan paralelismo real: aunque el SO asigna los hilos a distintos núcleos, el GIL los serializa a nivel del intérprete.

Los resultados de §3.2 cuantifican este efecto: V1 es 25–30× más lento que V0, lo que significa que cada evaluación con hilos cuesta el equivalente a 25–30 evaluaciones secuenciales en términos de tiempo de pared. El overhead tiene dos componentes: (1) el coste de creación y gestión del pool de hilos, que se amortiza a lo largo de la ejecución, y (2) el coste por tarea de adquisición y liberación del GIL y de los cambios de contexto del SO, que es constante y domina completamente para evaluaciones de microsegundos.

La situación se invierte radicalmente cuando las evaluaciones son I/O-bound. El GIL se libera automáticamente durante operaciones bloqueantes del sistema operativo: llamadas a `time.sleep()`, lectura de ficheros, operaciones de red, o cualquier extensión C que no tenga operaciones sobre objetos Python. En estas condiciones, los hilos sí pueden ejecutarse concurrentemente a nivel de sistema operativo, y V1 logra 18.5× de speedup en el experimento de latencia (§3.3).

4.2 Serialización IPC y el coste de V2 (multiproceso)

V2 evita el GIL completamente usando procesos independientes del SO, cada uno con su propio intérprete Python y su propia heap de memoria. Esto permite paralelismo CPU genuino: si hay N núcleos disponibles, N procesos pueden ejecutar código Python simultáneamente. Sin embargo, la independencia de memoria implica que el intercambio de datos entre el proceso coordinador y los workers debe hacerse mediante mecanismos de comunicación inter-proceso (IPC), que en Python se implementan serializando los objetos mediante pickle y transmitiéndolos a través de pipes del SO.

Para las funciones benchmark de este proyecto, cada evaluación completa en 1–10 µs (microsegundos). El coste mínimo de serialización pickle + pipe write + pipe read + deserialización para un array NumPy de 30 floats es del orden de 50–200 µs, es decir, entre 5 y 200 veces el tiempo de cómputo útil. V2 sería, en este escenario, entre 100× y 500× más lento que V0. El batching implementado en `MultiprocessEvaluator` mitiga parcialmente este problema agrupando varias partículas por transferencia IPC, pero no elimina el overhead fundamental. V2 resulta beneficioso únicamente cuando cada evaluación tarda **más de 10–50 ms**: simulaciones físicas, inferencia de modelos ML, renderizado, o cálculo de elementos finitos.

4.3 Vectorización NumPy y SIMD en V4

La vectorización de V4 representa un enfoque fundamentalmente distinto al paralelismo basado en concurrencia: en lugar de dividir el trabajo entre múltiples agentes de ejecución, elimina el overhead del intérprete Python del bucle de evaluación delegando el cómputo a rutinas de bajo nivel que operan sobre arrays contiguos en memoria. NumPy compila sus operaciones usando BLAS/LAPACK en la capa de álgebra lineal, y en arquitecturas x86 modernas genera automáticamente instrucciones vectoriales SIMD (SSE2, AVX, AVX-512) que procesan entre 4 y 16 flotantes en paralelo en un único ciclo de CPU.

El speedup de 3–5× observado en §3.2 tiene tres causas principales: (1) la eliminación del overhead del intérprete Python por cada evaluación (cada iteración del bucle Python cuesta ~50–100 ns de overhead puro); (2) la mejora en la localidad de caché, ya que NumPy organiza los datos en arrays contiguos que la CPU puede precargar eficientemente en la caché L1/L2; y (3) las instrucciones SIMD que permiten aplicar la

misma operación a múltiples datos simultáneamente sin ningún overhead de sincronización ni de IPC. El límite del speedup está determinado por la latencia de memoria y por el coste del paso de la barrera Python→C para iniciar cada operación.

El experimento de portafolio (§3.6) ilustra la limitación crítica de V4: si la función objetivo no implementa `fn_vec(positions)→ndarray`, `VectorisedEvaluator` ejecuta un fallback secuencial que no aporta ningún beneficio. La vectorización requiere que la función matemática sea expresable como operaciones NumPy sobre la matriz completa de posiciones, lo que es posible para funciones analíticas estándar pero no para funciones con estado, bifurcaciones complejas o llamadas externas.

4.4 Asyncio: cuándo la concurrencia cooperativa tiene sentido (V3)

El modelo de concurrencia de asyncio es fundamentalmente distinto al de los hilos: en lugar de preemption por el SO (el sistema operativo interrumpe los hilos en momentos arbitrarios), asyncio usa *cesión cooperativa* del control. Una corrutina ejecuta hasta que alcanza un punto `await` explícito, momento en que cede el control al event loop para que ejecute otra corrutina pendiente. Este modelo elimina por diseño las condiciones de carrera y la necesidad de locks explícitos para proteger estructuras de datos compartidas.

En el contexto de PSO, V3 es la estrategia natural para escenarios donde cada evaluación del fitness requiere operaciones de I/O asíncronas nativas: consultas HTTP mediante `aiohttp`, operaciones de base de datos mediante `asyncpg`, o cualquier biblioteca con interfaz `async/await`. En el experimento de §3.3, `AsyncEvaluator` usa `run_in_executor` porque `time.sleep()` es síncrono (la versión `async` sería `asyncio.sleep()`). Esto introduce el overhead de despachar tareas síncronas al executor interno, resultando en 12.9× vs 18.5× de V1. Si se usara `asyncio.sleep()` directamente, el event loop gestionaría el solapamiento sin overhead de executor y probablemente superaría a V1.

4.5 Reproducibilidad, logging y observabilidad

La reproducibilidad experimental es un requisito de primer orden en la evaluación de algoritmos estocásticos. El sistema garantiza reproducibilidad completa mediante: (1) control de semilla explícito en cada nivel (`numpy.random.default_rng(seed)` en el motor PSO y en la generación de datos de mercado), (2) almacenamiento de la semilla en cada fichero de resultados, y (3) instrumentación del entorno de ejecución en `meta.json` (plataforma, versión Python, número de CPUs, timestamp ISO 8601).

El historial por iteración guardado en CSV incluye cinco métricas: `best_fitness` (mejor conocido global), `mean_fitness` (media del enjambre), `std_fitness` (dispersión), `elapsed_eval` (tiempo de evaluación) y `elapsed_update` (tiempo de actualización de posiciones). Esta separación entre tiempo de evaluación y tiempo de actualización es deliberada: permite aislar el impacto de cada estrategia en la fase de evaluación sin contaminar la medición con el tiempo de actualización vectorizada, que es común a todas las estrategias. El logging usa el módulo estándar `logging` con niveles INFO/WARNING y formato estructurado (timestamp, módulo, mensaje), deshabilitado en la suite de tests para no contaminar la salida de `pytest`.

5. Conclusiones y recomendaciones

Este proyecto ha implementado y evaluado exhaustivamente cinco estrategias de evaluación paralela para PSO, cubriendo 180 ejecuciones benchmark, 405 combinaciones de hiperparámetros en grid search, un experimento de latencia asimétrica con V3, y un caso de uso real de optimización de portafolio financiero. Las conclusiones se organizan por el tipo de problema al que se enfrenta el usuario:

Escenario de evaluación	Estrategia óptima	Speedup esperado y razón
Función analítica vectorizable (sphere, rastrigin, ackley...)	V4 – Vectorización NumPy	3–5× sobre V0. Elimina el intérprete Python del bucle y aprovecha SIMD. Sin overhead de sincronización.

Evaluación con latencia I/O (API, red, BD, servicio externo)	V1 – Threading o V3 – Asyncio	V1: 18.5×, V3: 12.9×. El GIL se libera en I/O; concurrencia real. V3 preferible si la función es async nativa.
Evaluación CPU-bound costosa (simulación, modelo ML, > 50ms)	V2 – Multiprocessing	Único escenario donde IPC compensa. Usar batching para minimizar viajes. Speedup $\approx n_cores$.
Función no vectorizable ni con I/O (Python puro, < 1 μ s)	V0 – Secuencial	Todo overhead de paralelismo supera el tiempo de cómputo. V0 es la única opción racional.
Problema real sin fn_vec implementado (como el portafolio)	V0 – Secuencial	V4 cae a fallback con overhead adicional. V0 gana marginalmente ($\Delta=19.7$ ms). Implementar fn_vec para cambiar esto.

La conclusión transversal más importante es que no existe una estrategia universalmente óptima: la elección correcta depende del perfil de coste de la función objetivo (CPU vs I/O, duración por evaluación) y de si la función admite vectorización. El diseño modular del sistema, con la interfaz `Evaluator` como punto de variación, permite cambiar de estrategia sin modificar el algoritmo, lo que facilita la experimentación y la adaptación a nuevos problemas.

Todos los experimentos son reproducibles con las semillas indicadas. Entorno: Python 3.12.3 · CPython · WSL2 (Linux 6.6) · x86_64 · 1 vCPU. Semillas: {42, 123, 7, 99, 456}.